et al, 2022) attempts to fill this gap. There is also Elmokadem et al (2023) which provides a guide to using Stan/Torsten or Turing.jl (Ge et al, 2018) for Physiologically-based pharmacokinetic (PBPK) models. Given the above excellent resources, we identify the two main contributions of this paper as:

1. Offering a complete pharmacometrics-oriented Bayesian workflow including scripts and examples of common models using the Pumas software.
2. Offering an intuitive explanation of various Bayesian statistical and computational concepts that enable the fine tuning of algorithm's options and making sense of software outputs without being too technical.

We believe the Pumas Bayesian workflow implementation addresses many of the issues with other software by:

1. Providing a complete, user-friendly Bayesian analysis workflow that includes:
   (a) model and dosing regimen definition,
   (b) MCMC sampling,
   (c) diagnostics,
   (d) simulation,
   (e) counter-factual predictions, and
   (f) customizable cross-validation,
   using a few lines of code;
2. Using the same user-friendly, compact model syntax for hierarchical, dynamics-based models used in both Bayesian and non-Bayesian analyses so there is no need to translate models;
3. Using Julia as the core programming language which is a fast, interactive, and easy-to-use language, providing an excellent user experience;
4. Automating the definition and computational optimization of PMx models, e.g. using automatic parallelism over subjects and automatic ODE linearity and stiffness detection, delivering high performance with a neat syntax; and
5. Using the open-source, modular implementation, AdvancedHMC.jl (Xu et al, 2020), of the same MCMC algorithm that Stan uses, which is also used in Turing.jl (Ge et al, 2018)).

2.8 Paper Layout and Reading Plan

The rest of this paper is organized as follows. In Section 3, we describe the Bayesian workflow in Pumas. This should help users who are already familiar with the Bayesian workflow and theory to get started quickly using Pumas. In Section 4, we then a take a step back and give a reasonably comprehensive, intuition-oriented introduction to all of the main concepts and ideas invoked in the Bayesian workflow including the intuition behind the algorithm used and general points of caution when selecting priors. This section can help users make sense of highly technical concepts using mostly English and some light mathematics which can be useful to make informed decisions when using Pumas. Finally, Section 5 contains a number of common example models and Section 6 includes some additional reading material and resources for current and future Pumas users.

This paper is meant to serve both as a tutorial paper for users of Pumas and as an accessible resource for pharmacometricians learning Bayesian inference. If you are an experienced Bayesian and would like to learn how to use Pumas, then you can read Section 3 only and skip Section 4. If you are still learning Bayesian theory, we recommend reading at least some parts of Section 4 first. You can then use Table 1 to switch to reading the relevant parts in Section 3 if you prefer interweaving theory and code. For more focused tutorials and scripts, you can also check the Pumas tutorials website (tutorials.pumas.ai).

3 Bayesian Workflow in Pumas

A general Bayesian analysis workflow was presented in Gelman et al (2020b) which summarizes many of the current best practices in Bayesian statistics. There are many similarities between the general Bayesian workflow and the standard pharmacometrics workflow based on marginal MLE. There are a number of differences between them worth highlighting though:

1. The Bayesian workflow focuses on what a modeller can do when MCMC sampling fails for a model. This could be failure because the sampler is taking too long to complete its sampling, the sampler is often triggering numerical errors,or the samples are failing to pass the diagnostic tests after sampling. Things like simplifying the model or using less or more data can help pinpoint the issue and improve sampling. Much of

Theme	Section 3	Section 4
Model definition and prior choice	3.1	4.1, 4.2, 4.3
Prior and posterior predictions and simulations	3.2, 3.11-3.16	4.4
Inference algorithm and options	3.3-3.6	4.5-4.6
Convergence and diagnostics	3.7-3.10	4.7
Crossvalidation and model Selection	3.17-3.18	4.8

Table 1: This table shows the relevant groups of subsections from Section 3 and Section 4.

this is also applicable if MLE/MAP optimization fails but optimization failing is less common than MCMC failing with one of its many failure modes.
2. The Bayesian workflow includes prior selection diagnostics, e.g. prior predictive checks. Marginal MLE optimization does not use priors on population parameters, and there are well-established standard priors often used for patient-specific parameters so this is also less of an issue outside the Bayesian context.
3. The general Bayesian workflow mentions using the model and MCMC samples to make predictions, but it naturally does not specify what kinds of predictions which is very much context-specific. In pharmacometrics, we are typically interested in identifying the effect of using different dose amounts for new or existing patients or in predicting/simulating future responses given past data.

In this section, we will borrow components from the general Bayesian workflow, customize it for pharmacometrics, and present the Pumas syntax that can help a pharmacometrician follow the current best practices when performing Bayesian analysis. The syntax presented here is using Pumas 2.4 (to be released in June 2023), but Pumas 2.3 has most of the workflow implemented already. For updated syntax, options, and features, please refer to the Pumas documentation (docs.pumas.ai).

3.1 Defining a Model

3.1.1 Overview

In Pumas you can define models using the `@model` macro, which is composed of model blocks. We will cover six model blocks in this tutorial:

1. `@param`: where we define the population parameters of our model, along with their priors.
2. `@random`: where we define the subject-specific parameters of our model, along with their priors. This is an optional block which can be dropped for single subject models.
3. `@covariates`: where we declare the subject's covariates. This is an optional block which can be dropped for covariate-free models.
4. `@pre`: here we do all sorts of pre-computations and other statistical transformations, e.g. calculating the individual PK parameters.
5. `@dcp`: here you can define dose control parameters (DCPs) in the model. This is an optional block which can be dropped if no DCPs exist in the model.
6. `@dynamics`: where we define all of our dynamics, e.g. the system of ODEs that governs the relationship between the PK/PD compartments. This is an optional block which can be dropped for models without ODEs.
7. `@derived`: this is where we defined our observed response's distribution used to calculate the likelihood.
8. `@observed`: this is where we define additional variables to be computed during simulation but not fitting, e.g a non-compartmental analysis (NCA) based on the concentration curve can be performed here. This is an optional block.

With these blocks, you can code almost all PK/PKPD/PBPK models in Pumas. We'll cover the functionality of all of these model blocks.

First, the `@param` block is where we include all of the population parameters of our model. We begin by specifying a `begin ... end` where we insert one parameter per line. For each parameter, we give a prior with the $\sim$ operator followed by a distribution. Listing 1 shows an example of an `@param` block with three parameters and their priors. `tvcl` has a `LogNormal` prior with log-scale mean log(2.5) and unit log-scale standard deviation 1, `tvvc` has a positive-constrained `Normal`

prior with mean 70 and standard deviation 10, and σ has an `Exponential` prior with rate 3.[4]

```
@param begin
    tvcl ~ LogNormal(log(2.5), 1)
    tvvc ~ Constrained(Normal(70, 10); lower =
    0)
    σ ~ Exponential(3)
end
```

Listing 1: `@param` block example

Next, the `@random` block holds all of the subject-specific parameters and their priors. Similar to the `@param` block, we also begin by specifying a `begin ... end` where we insert one parameter per line; and each parameter is assigned a prior with the $\sim$ operator followed by a distribution. In Listing 2, we have an example of an `@random` block with a single parameter η which has an `MvNormal` (multivariate normal) prior with mean 0 and identity covariance matrix.

```
@random begin
    η ~ MvNormal([1 0; 0 1])
end
```

Listing 2: `@random` block example

The `@covariates` block is used to specify the subject's covariates. This is only a declaration block that follows the same approach by declaring one covariate per line inside the `begin ... end` statements. You can find an example in Listing 3 where we are declaring two subject covariates: `WT` for the subject's weight, and `SEX` for the subject's sex.

```
@covariates begin
    WT
    SEX
end
```

Listing 3: `@covariates` block example

We continue with the `@pre` block, where any pre-computations or statistical transformations necessary for our model are done. In this block, we can use all of the parameters and covariates declared in the previous blocks. The approach is similar to the other blocks so far: one pre-computation or transformation per line inside the `begin ... end` statements. Listing 4 provides an example of the `@pre` block, in which we compute the individual PK parameters for each subject.

```
@pre begin
    CL = tvcl * exp(η[1]) * (WT / 70)^0.75
    Vc = tvvc * exp(η[2]) * (WT / 70)
end
```

Listing 4: `@pre` block example

The fifth block, `@dynamics`, is where we specify the ODE system that governs the dynamics of our model. In this block, we declare one ODE per line inside `begin ... end` statements. On the left-hand side (LHS) of the equation is the derivative of the compartment, i.e. the rate of change. We are free to give each compartment any name. On the LHS the compartment name is followed with a `'` (prime) operator to denote the rate of change in that compartment. The prime operator is an intuitive way to declare the derivative of the compartment. On the right-hand side (RHS) of the equation is some combination of the compartments and the individual parameters specified in the `@pre` block. In the example in Listing 5, we specify the dynamics of the Theophylline model presented in section 2 with parameters `Ka`, `CL` and `Vc`..

```
@dynamics begin
    Depot' = -Ka * Depot
    Central' = Ka * Depot - CL/Vc * Central
end
```

Listing 5: `@dynamics` block example

Pumas supports automatic ODE linearity and stiffness detection. So even linear ODEs can be written in the above readable syntax with no performance loss. Additionally if ODE stiffness is detected, Pumas will switch to a stiff ODE solver as appropriate.

The `@derived` block is where we define our likelihood term. We can use two types of assignments in this block: the deterministic assignment `=` and the probabilistic assignment $\sim$. For the deterministic assignments, the RHS is a deterministic quantity, whereas for the probabilistic assignment the RHS is a probabilistic quantity represented as a distribution. In Listing 6, we have two variables being defined[5]. The first is `cp` defined using the deterministic assignment, and the second is the `conc` defined using the probabilistic assignment while also using `cp` as one of the parameters of the log-normal distribution. In this example, `conc` is observed and is called the dependent variable. We define the distribution of `conc` to be a log-normal distribution with log-scale mean log `cp` and

[4] To write LaTeX symbols like σ and θ in Pumas, you can write the LaTeX form, e.g. $\backslash sigma$, followed by the Tab key.

[5] We are using the macro `@.` to vectorize all of the function calls to the right of it. Since `Central` and `cp` are vectors of values at the observed time points.

```
@model begin
    @param begin
        tvcl ~ LogNormal(log(2), 0.25)
        tvvc ~ LogNormal(log(70), 0.25)
        tvka ~ LogNormal(log(1.5), 1)
        σ    ~ Constrained(Cauchy(), lower = 0)
    end
    @pre begin
        CL = tvcl
        Vc = tvvc
        Ka = tvka
    end
    @dynamics begin
        Depot'   = -Ka * Depot
        Central' =  Ka * Depot - (CL/Vc) * Central
    end
    @derived begin
        cp  := @. Central / Vc
        conc ~ @. LogNormal(log(cp), σ)
    end
end
```

Listing 8: PK 1-compartment single-subject model example

```
@model begin
    @param begin
        tvcl ~ LogNormal(log(2), 0.25)
        tvvc ~ LogNormal(log(70), 0.25)
        tvka ~ LogNormal(log(1.5), 1)
        σ    ~ Constrained(Cauchy(); lower=0)
        C    ~ LKJCholesky(3, 1.0)
        ω    ~ Constrained(
            MvNormal(
                zeros(3),
                Diagonal(0.4^2 * ones(3)),
            ),
            lower = zeros(3),
            upper = fill(Inf, 3),
            init=ones(3),
        )
    end
    @random begin
        η ~ MvLogNormal(Pumas.cor2cov(C, ω))
    end
    @pre begin
        CL = tvcl * η[1]
        Vc = tvvc * η[2]
        Ka = tvka * η[3]
    end
    @dynamics begin
        Depot'   = -Ka * Depot
        Central' =  Ka * Depot - (CL/Vc) * Central
    end
    @derived begin
        cp  := @. Central / Vc
        conc ~ @. LogNormal(log(cp), σ)
    end
end
```

Listing 9: Population PK 1-compartment first-order absorption model example

distributions for an all-new model, it will generally be a multi-step process consisting of:

1. **Deciding the support of the prior**. The support of the prior distribution must match the domain of the parameter. For example, different priors can be used for positive parameters than those for parameters between 0 and 1. Table 2 can help narrow down the list of options available based on the domain of the parameter.

Support	Distributions
$(0, 1)$	`Beta`, `KSOneSided`, `NoncentralBeta`, `LogitNormal`
$(0, \infty)$	`BetaPrime`, `Chi`, `Chisq`, `Erlang`, `Exponential`, `FDist`, `Frechet`, `Gamma`, `InverseGamma`, `InverseGaussian`, `Kolmogorov`, `LogNormal`, `NoncentralChisq`, `NoncentralF`, `Rayleigh`, `Weibull`
$(-\infty, \infty)$	`Cauchy`, `Gumbel`, `Laplace`, `Logistic`, `Normal`, `NormalCanon`, `NormalInverseGaussian`, `PGeneralizedGaussian`, `TDist`
Real vectors	`MvNormal`
Positive vectors	`MvLogNormal`
Positive definite matrices	`Wishart`, `InverseWishart`
Correlation matrices	`LKJ`, `LKJCholesky`
Other	`Constrained`, `truncated`, `LocationScale`, `Uniform`, `Arcsine`, `Biweight`, `Cosine`, `Epanechnikov`, `Semicircle`, `SymTriangularDist`, `Triweight`, `Pareto`, `GeneralizedPareto`, `GeneralizedExtremeValue`, `Levy`

Table 2: The table shows some of the most popular prior distributions available in Pumas and their corresponding support domains. You can learn more about each distribution using ? followed by the distribution name in the Pumas command line prompt.

2. **Deciding the center of the prior**, e.g. mean, median or mode.
3. **Deciding the strength of the prior**. This is often controlled by a standard deviation or scale parameter in the corresponding distribution function. A small standard deviation or scale parameter implies low uncertainty in the parameter value which leads to a stronger prior. A large standard deviation or scale parameter implies high uncertainty in the parameter value which leads to a weaker prior. It is recommended that each prior distribution that is considered to be used should be assessed carefully before using it. This will ensure that the strength of the prior reflects your confidence level in the parameter values. Refer to the discussion in Section 4.3 on prior selection for more details.
4. **Deciding the shape of the prior**. Some distributions are left-skewed, others are right skewed and some are symmetric. Some have heavier tails than others, e.g. the student's T-distribution is known for its heavier tail compared to a normal distribution. The shape of the probability density function (PDF) should reflect knowledge about the parameter value prior to observing the data.

When selecting new priors, besides the discussion in Section 4.3, you may also find the tips and recommendations in github.com/stan-dev/stan/wiki/Prior-Choice-Recommendations to be useful. For a more advanced discussion of various prior choices, also see Simpson et al (2014).

For univariate distributions, you can plot the distribution's probability density curve using the `PumasPlots.lines`[6] function, e.g:

```
using PumasUtilities
dist = Normal(0.0, 1.0)
PumasPlots.lines(dist)
```

For multivariate and matrix-variate distributions, you can use the `rand`, `mean` and `var` functions to sample from the distribution and make sense of the values' distributions. For example:

```
dist = LKJ(3, 1.0)
x = rand(dist, 100) # 100 samples
mean(x) # mean
var(x) # element-wise variance
```

The following is a description of some of the most popular prior distributions available in Pumas:

1. `Normal(`μ`, `σ`)`: univariate normal distributions with support $(-\infty, \infty)$, mean μ and standard deviation σ.
2. `LogNormal(`μ`, `σ`)`: univariate log normal distribution with support $(0, \infty)$ and a log-scale mean μ and log-scale standard deviation σ.
3. `MvNormal(`μ`, `Σ`)`: multivariate normal distribution with mean vector μ and covariance matrix

[6] which is a part of the `PumasUtilities` package

Σ. The matrix Σ can also be a diagonal matrix, e.g. `Diagonal([1.0, 1.0])`. You can also pass Σ alone as a matrix, e.g. `MvNormal(`Σ`)`, and the means will be assumed to be 0.

4. `MvLogNormal(`μ`, `Σ`)`: a multivariate log-normal distribution over positive vectors with log-scale mean vector μ and log-scale covariance matrix Σ as defined in the `MvNormal` case above.
5. `Cauchy(`μ`, `σ`)`: a univariate Cauchy distribution with support $(-\infty, \infty)$, location μ, and scale σ.
6. `Constrained(dist, lower = l, upper = u)`: a constrained prior distribution with a fixed support `(l, u)` and a fixed base distribution `dist` that could be any univariate or multivariate distribution. `lower` and `upper` set the lower and upper bounds on the random variables' support, respectively, defaulting to `-Inf` ($-\infty$) and `Inf` (∞), respectively. When `dist` is a univariate distribution, `lower` and `upper` should be scalars. When constraining multivariate distributions, `lower` and `upper` can be vectors or scalars. If set to a scalar, the same bound will be used for all random variables. There is also a `truncated` distribution which is different from `Constrained` in that it allows the base distribution to be a function of the model's parameters but `truncated` only supports univariate base distributions. In general, it's recommended to use `Constrained` in the `@param` block and `truncated` in the `@random` and `@derived` blocks. Examples:
   - `Constrained(Normal(0.0, 1.0), lower = 0.0)` is a half normal distribution.
   - `Constrained(Cauchy(0.0, 1.0), lower = 0.0)` is a half Cauchy distribution.
   - `Constrained(MvNormal([0.0, 0.0], [1.0 0.0; 0.0 1.0]), lower = 0.0)` is a constrained multivariate normal distribution.

   The `init` keyword argument can also be set to specify the initial value of the parameter, e.g. `Constrained(Normal(), lower = 0.0, init = 1.0)`
7. `truncated(dist, lower, upper)`: similar to `Constrained` with fixed lower and upper bounds `lower` and `upper`, respectively, and a base distribution `dist`. Setting `upper` is optional and it defaults to `Inf` (∞) when not set. In `truncated`, the base distribution `dist` is allowed to depend on the model's parameters and the normalization constant is computed in every log probability evaluation. However, the lower and upper bounds must be fixed constants and `truncated` only supports univariate base distribution. Examples: `truncated(Normal(0, 1), 0.0, Inf)` is a half normal distribution. `truncated(Cauchy(), 0.0, Inf)` is a half Cauchy distribution. `truncated(Normal(), -Inf, 0.0)` is a negative half normal distribution.
8. `Uniform(`l`, `u`)`: a univariate uniform distribution with lower and upper bounds l and u respectively.
9. `LKJ(`d`, `η`)`: a matrix-variate LKJ prior over correlation matrices of size $d \times d$. η is the positive shape parameter of the LKJ prior. Decreasing η results in samples with correlations closer to ± 1. There is also `LKJCholesky` which is semantically identical to `LKJ` but has some advantages. See below.
10. `LKJCholesky(`d`, `η`)`: a Cholesky-factorized version of the LKJ distribution where the matrix sampled is in factorized form. This is recommended over `LKJ` for use inside the model for performance reasons.
11. `Wishart(`ν`, `S`)`: a matrix-variate Wishart distribution over $d \times d$ positive definite matrices with ν degrees of freedom and a positive definite S scale matrix.
12. `InverseWishart(`ν`, `Ψ`)`: a matrix-variate inverse Wishart distribution over $d \times d$ positive definite matrices with ν degrees of freedom and a positive definite scale matrix Ψ.
13. `Beta(`α`, `β`)`: a univariate Beta distribution with support from 0 to 1 and shape parameters α and β.
14. `Gamma(`α`, `θ`)`: a univariate Gamma distribution over positive numbers with shape parameter α and scale θ.
15. `Logistic(`μ`, `θ`)`: a univariate logistic distribution with support $(-\infty, \infty)$, location μ and scale θ.
16. `LogitNormal(`μ`, `σ`)`: a univariate logit normal distribution with support $(0, 1)$ and a base normal distribution with mean μ and standard deviation σ.

The call syntax is `cv_res = crossvalidate(res, cv_method)`. To estimate the expected log predictive density (ELPD) given `cv_res`, you can use the `elpd` function: `elpd(cv_res)`.

There are 2 families of the crossvalidation algorithms available in Pumas:

1. Resampling-based CV which performs the MCMC again for each data point or subset left out. This algorithm is constructed using the `ExactCrossvalidation` struct.
2. PSIS-based CV (Vehtari et al, 2015) which uses the importance sampling and weight smoothing approach to avoid the need for resampling. This algorithm is constructed using the `PSISCrossvalidation` struct.

The constructor for a resampling-based CV method is:

```
cv_method = ExactCrossvalidation(; split_method,
    split_by, ensemblealg = EnsembleThreads())
```

where `split_method` and `split_by` are keyword arguments that must be set and `ensemblealg` defaults to the use of multi-threading.

This defines an instance of the `ExactCrossvalidation` algorithm for crossvalidation. In this algorithm, the fitting is re-run while leaving out a subset of the data each time.

The way by which the data is split between training and validation sets is determined using the keyword arguments `split_method` and `split_by`. The `split_method` argument can be of any of the following types:

1. `LeaveK` for leave-K-out crossvalidation
2. `KFold` for K-fold crossvalidation
3. `LeaveFutureK` for leaving K future points at a time

The `split_by` keyword argument can be of any of the following types:

1. `BySubject` for leaving out subjects
2. `ByObservation` for leaving out individual observations per subject

Each of the data-splitting methods will be discussed in the next subsection.

Similar to the resampling-based crossvalidation, the constructor for a PSIS-CV method is:

```
cv_method = PSISCrossvalidation(; split_method,
    split_by, pareto_shape_threshold = 0.7)
```

where `split_method` and `split_by` are keyword arguments that must be set.

This defines an instance of the `PSISCrossvalidation` algorithm for crossvalidation. The `split_method` and `split_by` keyword arguments are similar to the resampling-based CV case. The `pareto_shape_threshold = 0.7` keyword argument will result in the removal of any CV run that leads to a Pareto shape parameter more than 0.7 when computing the expected log predictive density (ELPD) estimate. This can be useful to avoid a few bad runs rendering the whole PSIS-CV method useless. Ideally one would re-run the inference for the subset of CV runs where the shape parameter exceeded the threshold but this is not implemented in Pumas yet as of the time of this writing. Follow the Pumas documentation (docs.pumas.ai) for updates on the latest features available.

3.17.1 Leave-K-Out

The constructor for the leave-K-out data splitting algorithm is:

```
split_method = LeaveK(; K = 5, shuffle = false,
    rng = nothing)
```

In this algorithm, the data is split multiple times into 2 disjoint groups, each time starting from the full data set. The 2 groups are typically called training and validation subsets, where the validation subset has K data points. In the next iteration, the whole data set is re-split using another disjoint subset of K data points as the validation subset. This process is done repeatedly until almost each data point has shown up in 1 and only 1 validation subset.

The data is typically a vector of some sort, e.g. observations or subjects, and the splittings are order-dependent. Before performing the splittings, you can randomly shuffle the data vector by setting the `shuffle` keyword argument to `true` (default is `false`) getting rid of the sensitivity to the original order of the data. You can additionally pass an optional pseudo-random number generator `rng` to control the pseudo-randomness for reproducibility.

Assume some dummy original data `["A", "B", "C", "D"]` which resembles the subjects or observations. Leave-one-out splitting without shuffling results in the data splittings shown in Table 3.

where each data point shows once and only once in a validation subset. Leave-2-out splitting without shuffling results in the data splittings shown in Table 4.

Training subset	Validation subset
["A", "B", "C"]	["D"]
["A", "B", "D"]	["C"]
["A", "C", "D"]	["B"]
["B", "C", "D"]	["A"]

Table 3: Leave-one-out splitting.

Training subset	Validation subset
["A", "B"]	["C", "D"]
["C", "D"]	["A", "B"]

Table 4: Leave-2-out splittings.

3.17.2 K-Fold

The constructor for the K-fold data splitting algorithm is:

```
split_method = KFold(; K = 5, shuffle = false,
    rng = nothing)
```

In this algorithm, the data is split K times into 2 disjoint groups, each time starting from the full data set. The 2 groups are typically called training and validation subsets, where the validation subset has `floor(N / K)`[9] data points, `N` being the total number of data points. In the next iteration, the whole data set is re-split using another disjoint validation subset of `floor(N / K)` different points, disjoint from the previous validation subsets. This process is done iteratively until almost each data point has shown up in 1 and only 1 validation subset. If `N` is divisible by `K`, each point will show up in 1 and only 1 validation subset. Otherwise, the remaining points will be part of the training subset for all the splittings and will not show up in any validation subset.

The data is typically a vector of some sort, e.g. observations or subjects, and the splittings are order-dependent. Before performing the splittings, you can randomly shuffle the data vector by setting the `shuffle` keyword argument to `true` (default is `false`) getting rid of the sensitivity to the original order of the data. You can additionally pass an optional pseudo-random number generator `rng` to control the pseudo-randomness for reproducibility.

Assume some dummy original data `["A", "B", "C", "D"]` which resembles the subjects or observations. 4-fold splitting without shuffling results in the data splittings shown in Table 5,

[9] `floor` is a function that rounds down to an integer.

Training subset	Validation subset
["A", "B", "C"]	["D"]
["A", "B", "D"]	["C"]
["A", "C", "D"]	["B"]
["B", "C", "D"]	["A"]

Table 5: 4-fold splittings.

where each data point showed once and only once in a validation subset. 2-fold splitting without shuffling results in the data splittings shown in Table 6.

Training subset	Validation subset
["A", "B"]	["C", "D"]
["C", "D"]	["A", "B"]

Table 6: 2-fold splitting.

3.17.3 Leave-Future-K

The constructor for the leave-future-K data splitting algorithm is:

```
split_method = LeaveFutureK(; K = 1, minimum =
    2)
```

In this algorithm, the data is assumed to be a time series. The goal is to split the data into "past" and "future". Using this algorithm, the data is split multiple times into 3 disjoint groups where the third group is discarded, each time starting from the full data set. The first 2 groups are typically called the past/training subset and the future/validation subset, where the future validation subset has `K` future data points. In the next iteration, the whole data set is then re-split using another disjoint subset of `K` data points as the future validation subset. This process is done iteratively starting from the full data set and moving backward in time until the training subset has less than a pre-set `minimum` number of points remaining. Using this method, each data point can show up in at most 1 future validation subset. The default values of `K` and minimum are 1 and 2 respectively.

Assume the original data is `["A", "B", "C", "D", "E", "F"]`. Leave-1-future-out splitting with `minimum = 2` results in the data splittings shown in Table 7 where the remaining points are discarded:

Leave-2-future-out splitting with `minimum = 2` results in the data splittings shown in Table 8.

Past subset	Future subset
["A", "B", "C", "D", "E"]	["F"]
["A", "B", "C", "D"]	["E"]
["A", "B", "C"]	["D"]
["A", "B"]	["C"]

Table 7: Leave-1-future-out splittings.

Past subset	Future subset
["A", "B", "C", "D"]	["E", "F"]
["A", "B"]	["C", "D"]

Table 8: Leave-2-future-out splittings.

3.17.4 Subject-based Splitting

The constructor for the subject-based splitting method is:

```
split_by = BySubject(; marginal = LaplaceI())
```

Using this method, each subject is treated as a single data point. The predictive log-likelihood computed for each subject can be either the marginal log-likelihood or conditional log-likelihood. This method has one keyword argument, `marginal`. If `marginal` is set to `nothing`, the predictive log-likelihood computed for each subject is the conditional log-likelihood using the typical values for the parameters. Otherwise, the predictive log-likelihood computed for each subject is the marginal log-likelihood using `marginal` as the marginalization algorithm. The default value of marginal is `LaplaceI()` which uses the Laplace method to integrate out the subject-specific parameters. Other alternatives include: `FOCE()` and `FO()`.

3.17.5 Observation-based Splitting

The constructor for the observation-based splitting method is:

```
split_by = ByObservation(; allsubjects = true)
```

Using this method, each observation or collection of observations is treated as a single data point. When computing predictive log-likelihoods using this method, the predictive log-likelihood computed is the conditional log-likelihood of one or more observations for one or more subjects.

This method has one keyword argument, `allsubjects`. If `allsubjects` is set to `true` (the default value), the i^{th} observation of each subject are all grouped together into a single data point. This assumes all subjects have the same number of observations. If `allsubjects` is set to false, then each observation for each subject is its individual data point.

Assume there are 2 subjects and 3 observations per subject. When using `split_method = LeaveK(K = 1)` as the splitting method together with `split_by = ByObservation(allsubjects = false)`, the training and validation splittings are shown in Table 9.

Training subset	Validation subset
Subj 1 (obs 1, 2, 3), subj 2 (obs 1, 2)	Subj 2 (obs 3)
Subj 1 (obs 1, 2, 3), subj 2 (obs 1, 3)	Subj 2 (obs 2)
Subj 1 (obs 1, 2, 3), subj 2 (obs 2, 3)	Subj 2 (obs 1)
Subj 1 (obs 1, 2), subj 2 (obs 1, 2, 3)	Subj 1 (obs 3)
Subj 1 (obs 1, 3), subj 2 (obs 1, 2, 3)	Subj 1 (obs 2)
Subj 1 (obs 2, 3), subj 2 (obs 1, 2, 3)	Subj 1 (obs 1)

Table 9: Training and validation splits using `split_method = LeaveK(K = 1)` and `split_by = ByObservation(allsubjects = false)`.

On the other hand, if `allsubjects` is set to `true`, the training and validation splittings are shown in Table 10.

Training subset	Validation subset
Subj 1 (obs 1, 2), subj 2 (obs 1, 2)	Subj 1 (obs 3), subj 2 (obs 3)
Subj 1 (obs 1, 3), subj 2 (obs 1, 3)	Subj 1 (obs 2), subj 2 (obs 2)
Subj 1 (obs 2, 3), subj 2 (obs 2, 3)	Subj 1 (obs 1), subj 2 (obs 1)

Table 10: Training and validation splits using `split_method = LeaveK(K = 1)` and `split_by = ByObservation(allsubjects = true)`.

3.17.6 Examples

Assume there are 5 subjects and 10 observations per subject and that `res` is the result of the `fit` or `discard` function. The following are some of the combinations in which the above inputs can be used:

- Leave-one-observation-out cross-validation, leaving 1 observation for all the subjects at a time.

`allsubjects = true` means that the same observation index is removed for all the subjects, e.g. the 10th observation for all the subjects is used for validation in the first run, then the 9th observation is used for validation in the second run, etc.

```
split_method = LeaveK(K = 1)
split_by = ByObservation(allsubjects = true)
cv_method = ExactCrossvalidation(;
    split_method = split_method, split_by =
    split_by, ensemblealg = EnsembleThreads()
    )
cv_res = crossvalidate(res, cv_method)
```

- Leave-two-future-observations-out cross-validation, leaving 2 future observations per subject such that no less than 4 observations are used in training. `allsubjects = false` means that only the data of one subject at a time will be split. So in the first run, observations 1 to 8 of subject 1 are used for training, and observations 9 and 10 of subject 1 are used for validation. In the second run, observations 1 to 6 of subject 1 are used for training, and observations 7 and 8 are used for validation. In the third run, observations 1 to 4 of subject 1 get used for training, and observations 5 and 6 are used for validation. For all 3 runs, all the observations of subjects 2 to 5 are used in training. Then in the fourth to sixth runs, subject 2's data gets split. In the seventh to ninth runs, subject 3's data gets split, etc.

```
split_method = LeaveFutureK(K = 2, minimum =
    4)
split_by = ByObservation(allsubjects = false)
cv_method = ExactCrossvalidation(;
    split_method = split_method, split_by =
    split_by, ensemblealg = EnsembleThreads()
    )
cv_res = crossvalidate(res, cv_method)
```

- Leave-one-subject-out cross-validation, marginalizing the subject-specific parameters using FOCE when computing predictive likelihoods. Using this method, subjects `1, 2, 3, 4` are used for training in the first run and subject `5` is used for validation. In the second run, subjects `1, 2, 3, 5` are used for training and subject `4` is used for validation, etc. The predictive likelihood computed is the marginal likelihood of the subject left out, computed using `FOCE` for marginalizing the subject-specific parameters. `LaplaceI` or `FO` could have also been used instead.

```
split_method = LeaveK(K = 1)
split_by = BySubject(marginal = FOCE())
cv_method = ExactCrossvalidation(;
    split_method = split_method, split_by =
    split_by, ensemblealg = EnsembleThreads()
    )
cv_res = crossvalidate(res, cv_method)
```

3.18 Information Criteria

The ELPD model evaluation metric computed from the crossvalidation output is theoretically similar to the so-called Widely Applicable Information Criteria (WAIC) (Vehtari et al, 2015). More precisely, -2 times the ELPD is comparable to the WAIC. A higher ELPD Is better and a lower WAIC is better. When the ELPD is estimated using the PSIS leave-one-out (LOO) crossvalidation method, -2 times the ELPD estimate is sometimes known as the LOOIC. Besides the ELPD estimates, Pumas also supports common information criteria (Burnham and Anderson, 2002), such as the Akaike information criteria (AIC), the corrected AIC (AICc), the Bayesian information criteria (BIC), and the WAIC. To estimate these, we first need to compute the pointwise log-likelihoods by some definition of "pointwise". To do this, you can call:

```
pl = loglikelihood(tres; split_method, split_by)
```

where `tres` is the output of `fit` or `discard` and `split_method` and `split_by` are the keyword arguments explained in Section 3.17 which define what a point is and which log-likelihood to compute. To calculate the information criteria using the pointwise log-likelihoods, you can then use any of the following functions:

```
Pumas.aic(pl)
Pumas.aicc(pl)
Pumas.bic(pl)
Pumas.waic(pl)
```

4 Background and Intuition

In this section, the notation used and the mathematical background of Bayesian inference in pharmacometrics will be presented. We include a brief introduction of Bayesian statistics and where it's useful in pharmacometrics, followed by an intuitive explanation of Markov Chain Monte Carlo (MCMC) and the No-U-Turn sampler (NUTS) algorithm (Hoffman and Gelman, 2014; Betancourt, 2017). We then discuss the intuition and some of the math behind prior selection, MCMC convergence diagnostics, and cross-validation and model selection. This section should prepare the readers

2. Weak priors, i.e. weakly informative priors.

These are loose categories because in reality, only the relative strength of a prior compared to the likelihood is significant. Recall that the joint probability used to drive the MCMC sampling is given by:

$$p(D, \eta, \theta) = p(D \mid \eta, \theta) \times p(\eta, \theta) \tag{11}$$

which is the product of the prior probability and likelihood value. If a lot of data exist, the likelihood term will dominate most priors and the posterior distribution will be mostly reflective of the parameter values that fit the data well. If not enough data exist, the prior and its choice become more significant since it's possible some choices of the prior will lead it to dominate the above term. When the prior dominates the likelihood, the posterior distribution will be only a slight shift of the prior distribution towards the parameter values that *actually* fit the data well. In these cases, the danger of abusing the prior in analyses is higher and we will discuss scenarios where that can happen in this section.

Harmless priors are priors that largely mimic the purpose of specifying parameter domains in non-Bayesian workflows. These priors have very little to no preference between parameter values and they can be easily overpowered by even a few data points. For instance, a uniform distribution over $[0, 10^{10}]$ for a positive parameter can be considered a harmless prior. This prior only encodes the domain of the parameter without favouring certain values over others in this particular parameterization.

Good and bad priors share one thing in common: they are both "*informative*". More precisely, good priors are informative and bad priors are misinformative. Table 11 summarizes the various types of informative and mis-informative priors. A good informative prior is one that has a good scientific basis and doesn't contradict the data.

Given Table 11, there are 2 general ways to define sound priors:

1. Define a weakly informative prior[18] and ensure that the conclusion of the study does not change in the limit as the prior gets weaker and weaker. This is a case where we are letting the data speak for itself without imposing our own bias on the results. In this case, we are only using Bayesian inference for its probabilistic soundness and ability to quantify the total uncertainty in the parameters and response even when the model is non-identifiable and/or we only have a few data points.
2. Use similar previous studies to guide the prior choice and test that it doesn't contradict the new data. In this case, more justification of the prior distribution is necessary and a proof that the prior doesn't contradict the data is required.

One way to show that the prior doesn't contradict the data is to start with no observations at all and with a weakened version of the strong prior, e.g. by increasing the standard deviation. You can then incrementally make the prior stronger again (e.g. by decreasing the standard deviation back) until it reaches the desired level of strength, followed by incrementally adding the (previously removed) observations back to the analysis. If doing so and re-running the analysis at every increment shows a consistent trend in all of the significant statistics (e.g. the probability that the drug is effective), then the prior's strength is aligned with the story told by the data. This can be done using a sequence of prior simulations (when all the data is removed) followed by a combination of MCMC runs and posterior simulations (when the data is added back). For more methods for detecting prior-data conflicts and model mis-informativeness, the readers are referred to Kallioinen et al (2021).

Another simple way to detect the potential conflict between data and informative priors is to simulate from the following distribution:

$$\begin{aligned} (\eta, \theta) &\sim p(\eta, \theta) \\ y &\sim p(y \mid \eta, \theta) \end{aligned} \tag{12}$$

where $p(\eta, \theta)$ is the prior distribution. You can then do a simulation or VPC plot checking the consistency of the data and prior simulations. If the prior is weakly informative or nearly non-informative and we have a lot of data such that the likelihood dominates the prior, prior simulations that are inconsistent with the data in a VPC plot can be ignored. However if the prior is informative, it is based on previous studies and there are not enough data points in the new study to fully dominate the prior, the VPC plot of the prior simulations next

[18] Very weak priors are sometimes called diffuse priors.

Scientific Basis	Contradicts Data?	Comment
None	Yes	This is cheating. Using a strong prior that contradicts and over-powers the data with no scientific basis can be used to cook any results we desire. For instance, a drug can be ineffective but if the strong prior says it's effective with a very high probability and not enough data exists to counter that strong wrong prior, the conclusion of the "analysis" will be that the drug is effective.
None	No	This is bad science. Using a strong prior that's consistent with the data but that over-powers the likelihood with no scientific basis can lead to over-confident predictions and premature conclusions using fewer data points than necessary. This has a similar effect to artificially replicating the data multiple times to artificially reduce the confidence interval in the non-Bayesian workflow.
Previous studies	Yes	This is a sign that dis-similar previous studies were used to guide the prior choice and that the prior choice should be revised because it contradicts the new data and can possibly bias the results of the analysis.
Previous studies	No	When used with care, results from previous studies (e.g. an approximation of its parameters' posterior distribution) can be used to guide the selection of good informative priors for a new similar study. These priors should not contradict the new data collected but may help us terminate the new study early using a smaller sample size than otherwise possible. Positively concluding a study early, after it's become clear that a drug is effective given all the information available, means that more patients can have access to *truly effective* drugs earlier. This is especially important for rare disease drug development where collecting more data in a study often means adding years to the clinical trial duration. This is a use case that requires more regulatory and industry agreement on best practices for defining informative prior distributions in such studies with the goal of benefiting the patients.

Table 11: The table shows a description of a number of ways to choose **informative prior distributions**. Only the last case is a good use of informative priors. **This table is only applicable to informative priors** that may dominate the likelihood, since weakly informative priors that are dominated by the likelihood typically don't matter as much.

to the data can be a useful diagnostic to inspect. Refer to Section 3.2 for how to do this in Pumas.

When using a weakly informative prior, you may be inclined to use such simulation plots to select a good prior with a good (but not necessarily tight) coverage of the data. In general, any fine-tuning of the prior based on the data is frowned upon. This is because we would then be using the same data twice, once to fine-tune the prior and once to update the prior to get the posterior. This can result in under-estimating the posterior's variance, i.e. over-confident posteriors, which in turn leads to over-confident posterior predictions. This is analogical to replicating some or all of the data points in your data in a frequentist workflow which under-estimates the confidence intervals and standard errors.

In cases where sampling fails due to numerical errors, an overly weak prior may be a cause. In this case, one may try changing the prior, e.g. truncating its support to reasonable bounds, to be more consistent with the data. However, only minimal

such changes can be allowed in the final analysis and a good post-sampling sensitivity analysis study would be needed to ensure that the conclusion of the study is not locally sensitive to the prior. More generally, numerical errors are often a sign that the model is too sensitive to some of the parameter values which may imply a structural problem in the model itself. In this case, one should also consider better fixes than simply truncating the priors, e.g. by simplifying the model or re-parameterizing it to avoid numerical instability. The Bayesian workflow paper by Gelman et al (2020b) has excellent recommendations and case studies to take heed from when diagnosing failing MCMC runs.

4.3.3 Correlation vs Covariance

When defining models that have a covariance matrix parameter (e.g. the covariance parameter of the multivariate normal prior distribution typically used for subject-specific parameters in pharmacometrics), one is always faced with the following 2 equivalent parameterizations:

1. Use a covariance matrix parameter Ω, or
2. Use a vector of standard deviations ω and a correlation matrix parameter C.

One can easily recover Ω from (ω, C) and vice versa. Let D_ω be the diagonal matrix whose elements are ω, ω_i be the i^{th} element in ω, and $\Omega[i,i]$ be the i^{th} diagonal element of Ω. The relationships between C, ω and Ω is given by:

$$\begin{aligned} \Omega &= D_\omega \times C \times D_\omega \\ C &= D_\omega^{-1} \times \Omega \times D_\omega^{-1} \\ \omega_i &= \sqrt{\Omega[i,i]} \end{aligned} \tag{13}$$

In the non-Bayesian context, the 2 parameterizations are equivalent. However in Bayesian analysis, one should define a prior distribution on the parameters. Since prior distributions are supposed to encode the domain knowledge and state of belief about the values of the parameters, using more intuitive/interpretable parameterizations is generally recommended to better make sense of the prior distributions used. For this reason, some people prefer to use the second parameterization with separate standard deviations vector and a correlation matrix since they are more interpretable. We saw examples of prior distributions that can be used for standard deviation, correlation and covariance matrix parameters in Section 3.

4.4 Markov Chain Monte Carlo (MCMC) Intuition

In this section, the use of MCMC will be motivated showing how MCMC can be used to bypass the need for high dimensional integrals (discussed in Section 4.2) for all practical purposes.

4.4.1 Inference

Markov Chain Monte Carlo (MCMC) (Brooks et al, 2011) bypasses the need to solve the numerical integration problem by sampling[19] from the posterior probability distribution $p(\eta, \theta \mid D)$ directly using only the tractable numerator in Eq 7. This numerator is sometimes called the *joint* probability since it is just $p(D, \eta, \theta) = p(D \mid \eta, \theta) \cdot p(\eta, \theta)$. Note the difference between the terms with and without conditioning $|$. Having samples from the posterior allows us to estimate quantities such as:

$$p(f(\eta, \theta) > 0 \mid D) \tag{14}$$

for an arbitrary function f using a simple count by checking all the samples from the posterior and counting how many satisfy the particular conditions of interest. The ratio of samples satisfying the condition $f(\eta, \theta) > 0$ is the unbiased estimate of the above probability. More concretely, this probability could be $p(\theta_i > 0 \mid D)$ where θ_i corresponds to the effect size of an experiment treatment arm compared to the placebo arm.

4.4.2 Prediction

Besides estimating probabilities of events using the posterior samples, the posterior samples can also be used to make predictions. The functions for performing posterior predictions in Pumas were

[19] A sample of a random variable or a sample from its distribution is an instantiation of said random variable. For instance, $[0, 1, 0, 0, 1, 1]$ are 6 samples from a Bernoulli distribution whose support is the set $\{0, 1\}$. The support of a distribution is the domain of the random variable. For example, the set $\{0, 1\}$ is the support of the Bernoulli distribution and the set of all real numbers is the support of the normal distribution. A sample from the posterior distribution can be interpreted as the likely parameter values that could have generated the data that we observed. The term *sample* can sometimes be used to refer to multiple such samples, to be understood from the context.

posterior, the NUTS algorithm uses a multi-step proposal mechanism with a fixed time step size (determined during the adaptation phase and then fixed) and a dynamic number of time steps (dynamic in both the adaptation and sampling phases). More specifically, the sampler simulates a trajectory of T time steps before choosing a proposal randomly from this trajectory, where T is different for each proposal made.

The number of time steps T simulated by NUTS is determined by an incremental simulation of: $T = 1+2+4+8+16+\ldots$ time steps where the number of time steps in each incremental simulation is an increasing power of 2. Each incremental simulation can be either:

1. Forward in time starting from the future-most state, or
2. Backward in time starting from the past-most state.

The direction of each incremental simulation is sampled randomly with 0.5 probability assigned to each direction. Table 12 shows an example of the incremental simulations for the particular choice of simulation directions: [Forward, Forward, Reverse, Reverse, Forward, Reverse].

So when do we stop simulating? The NUTS algorithm typically stops simulating when one of the following 4 conditions is met:

1. It finds a so-called U-turn, that is when the sampler begins to move back towards one end of the trajectory from the other end.
2. It reaches a pre-set maximum number of simulation steps.
3. The log prior probability and/or log likelihood drops rapidly in one of the steps, dropping by more than a pre-set threshold.
4. A numerical error occurs.

The third and fourth termination criteria are often called "divergence".

After the simulation terminates, a point randomly chosen on the trajectory simulated becomes the next proposal and the search is terminated. *Terminating by finding a U-turn is typically considered a sign of successful exploration.* The number of evaluations of $\log p(D, \eta, \theta)$ in each NUTS iteration is determined by the length of the simulated trajectory which is $\sum_{i=0}^{j-1} 2^i = 2^j - 1$, if j incremental simulations were required to find a U-turn[32].

In the efficient implementations of the NUTS algorithm, a binary tree data structure of depth j is used to help with the efficient selection of a proposal from all of the states (η, θ) visited during the simulation until a U-turn was found[33], without storing all of the states. This is an efficiency enhancement trick but the term *tree depth* stuck and became synonymous to the number of incremental simulations ran so far, j. In the case where the sampler is unable to find a U-turn even after a pre-specified maximum j is reached, the sampler terminates the simulation anyways and makes a proposal. The term maximum tree depth is commonly used to refer to the maximum number of incremental simulations j allowed before having to make a proposal even if no U-turn was found.

4.5.7 Optional: Distribution of the Initial Velocity

Recall that in each NUTS iteration, we are sampling a random initial velocity for the (η, θ) particle before simulating the dynamics to arrive at a proposal. Hypothetically, assume that we already have samples from the posterior $p(\eta, \theta \mid D)$. If you were to go back and re-do the sampling using NUTS, how would you sample the initial velocity of the imaginary (η, θ) particle to make sampling as efficient as possible? In general, it would make sense to move faster along directions in the posterior that have a higher variance and slower along directions that have a lower variance. For instance, we can compute the variance along each parameter's axis and sample higher speeds for the parameters that change more, and lower speeds for the parameters that change less. In practice, you can think of different parameters having different scales where 1 parameter may be in the 10s while another one may be in the 1000s. In that case, it makes sense

[32] In the efficient implementation of NUTS, once a U-turn is found, the final incremental simulation is interrupted so the number of model evaluations is actually somewhere between 2^{j-1} and $2^j - 1$

[33] The number of states visited excluding the initial state is at most $2^j - 1$. Adding the initial state, we have 2^j possible states any of which could be the proposal. These can in theory be stored as the leaf nodes of a binary tree of depth j which has 2^j leaf nodes. However in practice, only a subset of such states are stored and the tree idea is used to ensure the proposal can be efficiently chosen at random from all 2^j possible states while satisfying the theoretical requirements of a proposal in MCMC, which is often called the detailed balance condition.

Increment j	Simulation Direction	Interval of the Time Steps Simulated after j Increments
0	-	$[0, 0]$
1	Forward	$[0, 0+1] = [0, 1]$
2	Forward	$[0, 0+1+2] = [0, 3]$
3	Reverse	$[0-4, 0+1+2] = [-4, 3]$
4	Reverse	$[0-4-8, 0+1+2] = [-12, 3]$
5	Forward	$[0-4-8, 0+1+2+16] = [-12, 19]$
6	Reverse	$[0-4-8-32, 0+1+2+16] = [-44, 19]$

Table 12: The table shows the incremental simulations of the NUTS algorithm for $j \in [1, 6]$. Notice how an increasing power of 2 is added to the positive direction or subtracted from the negative direction in each increment. The total number of time steps made after increment j (excluding the initial time point $t = 0$) is $1 + 2 + 4 + 8 + 16 + \cdots = 2^0 + 2^1 + 2^2 + \cdots + 2^{j-1} = \sum_{i=0}^{j-1} 2^i = 2^j - 1$. **Check**: $2^1 - 1 = 2^0 = 1$, $2^2 - 1 = 2^0 + 2^1 = 3$, $2^3 - 1 = 2^0 + 2^1 + 2^2 = 7$, etc. Note that the intervals above are of the number of time steps. Each time step has a simulated time step size of ϵ.

to use different speeds along different directions to more efficiently sample from the posterior distribution. More generally, one may even compute the sample covariance matrix from the (hypothetical) samples available, compute the principal components and sample higher speeds along directions with more variance than the other directions.

If we encode how slow we want the particle to go along each direction d_i by a number s_i, setting the standard deviation of the speed along this direction to $1/s_i$ can be used to achieve the desired slowness. Assume each d_i is an axis along a specific parameter i (which could be part of η or θ). The distribution of the velocity v_i along d_i can be the following univariate Gaussian:

$$v_i \sim N(0, (1/s_i)^2) \tag{17}$$

with mean 0 and standard deviation $1/s_i$. This distribution will have us sampling speeds along the direction d_i that are on average inversely proportional to s_i.

Writing it for all the parameters together, we can write:

$$v \sim N(0, M^{-1}) \tag{18}$$

where M is a diagonal matrix of elements s_i^2 on the diagonal and M^{-1} is the covariance matrix of the velocity vector v. Using a diagonal M is equivalent to adapting the speeds' standard deviations along the parameters' axes. While using a dense matrix M is equivalent to the more general case of adapting the speeds' standard deviations along more optimized directions d_i (e.g. from principal components of the covariance matrix).

It turns out that when simulating the "imaginary dynamics" in HMC/NUTS after sampling the initial velocity, the analogical *kinetic energy* is given by:

$$K(v) = v^T M v/2 \tag{19}$$

hence the natural inclination to call the above matrix M a "mass matrix" in the HMC/NUTS literature. Recall that in physics, the kinetic energy of a particle with a scalar speed v and mass m is $\frac{mv^2}{2}$.

To summarize, directions with a higher "mass" will be explored more slowly than directions with a lower mass. The ideal mass matrix M is one that approximates the *global* precision matrix of the posterior distribution, i.e. the inverse of the covariance matrix. Equivalently, the ideal M^{-1} is one that approximates the global covariance matrix of the posterior.

So far we assumed that we have samples from the posterior and are able to adapt the mass matrix manually. In practice, the NUTS algorithm adapts the mass matrix for you during the adaptation phase, and you only need to select the structure of the matrix, e.g. diagonal or dense. For large problems, a diagonal matrix is typically used in practice since the computational cost of using a dense matrix is $O(D^3)$, where D is the total number of parameters in (η, θ) combined. On the other hand, the computational cost of using a diagonal

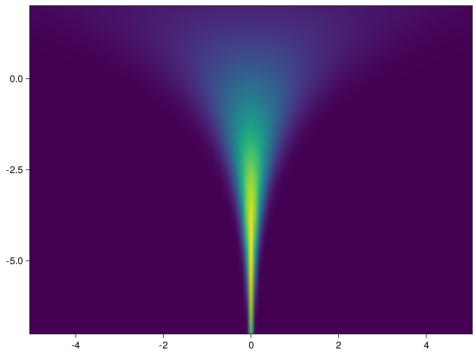

Figure 16: The PDF heatmap of the prior distribution of $\log\omega$ on the y-axis and η on the x-axis.

matrix is only $O(D)$. When we have many subjects in the hierarchical model, D can be quite large.

Before we conclude this section, it is important to note that the HMC/NUTS algorithm is typically explained in the literature using a so-called momentum vector $p = Mv$ while we chose to use the more intuitive velocity vector v in this paper to explain the intuition behind the algorithm. The two parameterizations are equivalent but the momentum one is the one typically used in implementations and HMC/NUTS research. When $v \sim N(0, M^{-1})$, the corresponding distribution of the momentum vector p is $N(0, M)$.

4.5.8 Optional: Hierarchical Priors and NUTS

Consider the following toy model which has no observations:

$$\begin{aligned} \log\omega &\sim \text{Normal}(0, 1.5) \\ \eta &\sim \text{Normal}\left(0, (e^{\log\omega})\right) \end{aligned} \tag{20}$$

This is a model with 2 parameters $(\log\omega, \eta)$, a prior distribution on each of them and some exponential dependence between them in that the standard deviation of the prior on η depends exponentially on the value of $\log\omega$. Figure 16 shows the PDF heatmap of the joint prior distribution of the parameters $\log\omega$ (y-axis) and η (x-axis). Recall that the NUTS algorithm uses a multi-step trajectory with a fixed time step size in the imaginary dynamics simulation to account for variation in curvature along the same direction. Consider the direction along the x-axis in the figure. The curvature along the x-axis changes depending on where along the y-axis the imaginary $(\log\omega, \eta)$ particle is. Lower values of $\log\omega$ lead to exponentially higher curvatures (reflected through the tight color band in the heatmap) along the η direction. So if we try to use NUTS to sample from this prior, two bad things can happen:

1. The sampler may adapt its step size to very small values to be able to sample from the lower regions of the prior and it will use a large number of time steps T to effectively explore the upper regions of the prior. In such cases, more often than not, the maximum number of allowed time steps $2^j - 1$ may not be enough to find the U-turn. This will hurt the performance since we will be doing many model evaluations per proposal and we may need multiple steps to fully traverse the upper regions of the prior.
2. The sampler may adapt its step size to values not small enough to sample from the lower regions in the prior. In this case, the sampler may skip sampling from the significant lower part in the prior leading to potential bias in the results.

In other words, the above prior may lead to slow and biased NUTS sampling, a clearly terrible outcome.

Note that of course we are not trying to sample from the prior using NUTS, because we can just sample directly from the standard distributions in the model using more direct and efficient methods. However, studying the prior's PDF and how it interacts with NUTS can help us understand how NUTS will interact with the posterior when there are a few data points available. Also note that the above model is using an explicit log scale parameterization for the standard deviation for pedagogical purposes. In reality, models may be written directly in terms of the standard deviation ω (instead of its log) or more generally the covariance matrix Ω for multivariate η. However, the above example is still relevant in those cases because implementations of the NUTS algorithm do the log-scale transformation behind the scenes and so NUTS actually samples unconstrained parameters all the time even if the original model's parameters were constrained to a different support. So the same insight we build for the model above is applicable to general hierarchical models when using NUTS.